Introducción a Docker y CI/CD con GitHub

Contenedores, automatización y primeros pipelines para el curso

Material didáctico para alumnos

Curso de desarrollo y tecnologías software

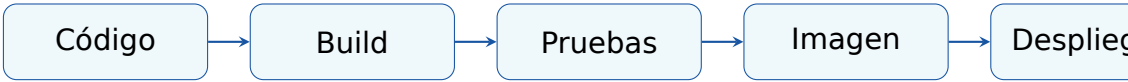
2026

1. Objetivos de la sesión

- Comprender qué problema resuelve Docker.
- Diferenciar imagen, contenedor, volumen, red y registro.
- Crear una imagen sencilla mediante un `Dockerfile`.
- Ejecutar y probar una aplicación en un contenedor.
- Entender CI, CD y el concepto de pipeline.
- Crear un primer workflow con GitHub Actions.
- Relacionar Docker + GitHub + pruebas automatizadas.

Idea central: **crear software reproducible y automatizar su validación** antes de llevarlo a otro entorno.

2. Del código local al software entregable



- El problema no termina cuando “funciona en mi ordenador”.
- Necesitamos entornos reproducibles y una forma automática de comprobar los cambios.
- Docker ayuda a empaquetar; CI/CD ayuda a automatizar.

3. ¿Qué es Docker?

- Plataforma para desarrollar, distribuir y ejecutar aplicaciones mediante contenedores.
- Un contenedor proporciona un entorno aislado para ejecutar una aplicación.
- La aplicación y sus dependencias viajan juntas en una imagen.
- El objetivo educativo: reducir diferencias entre desarrollo, pruebas y despliegue.

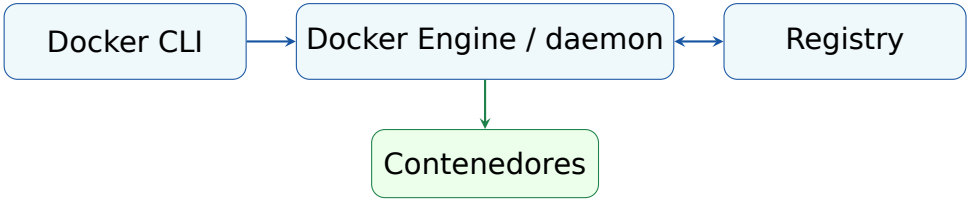
Un contenedor no es una máquina virtual completa: comparte el kernel del sistema anfitrión y proporciona aislamiento a nivel de procesos.

4. Contenedores frente a máquinas virtuale

	Máquina virtual	Contenedor
Unidad	VM completa	Proceso aislado
Sistema operativo	SO invitado	Comparte kernel del host
Arranque	Más pesado	Normalmente más rápido
Aislamiento	Virtualización	Aislamiento de procesos
Uso típico	Infraestructura completa	Aplicaciones y servicios

Importante: no son tecnologías excluyentes; pueden utilizarse juntas.

5. Arquitectura básica de Docker



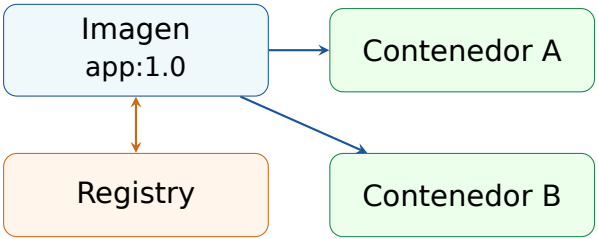
- El cliente envía órdenes al motor Docker.
- El motor gestiona imágenes, contenedores, redes y volúmenes.
- Un registry almacena y distribuye imágenes.

6. Los cinco conceptos que debes dominar

1. **Imagen:** plantilla inmutable para crear contenedores.
2. **Contenedor:** instancia ejecutable de una imagen.
3. **Dockerfile:** instrucciones para construir una imagen.
4. **Volumen:** mecanismo para persistir datos fuera del ciclo de vida del contenedor.
5. **Red:** permite la comunicación entre contenedores y con otros sistemas.

Regla mental: **Dockerfile** construye una **imagen**; la imagen se ejecuta como **contenedor**; los datos persistentes van a **volúmenes**.

7. Imagen, contenedor y registro



- Una misma imagen puede servir para crear múltiples contenedores.
- Etiquetar versiones facilita reproducibilidad: `app:1.0`, `app:1.1`, etc.

8. ¿Qué es un Dockerfile?

- Es un fichero de texto que describe cómo construir una imagen.
- Define una imagen base, copia código, instala dependencias y configura el arranque.
- Cada instrucción contribuye a una capa de la imagen.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

La imagen debe contener lo necesario para ejecutar la aplicación, pero no secretos ni datos que deban persistir.

9. Construir y ejecutar una imagen

```
# Construir
docker build -t mi-app:1.0 .

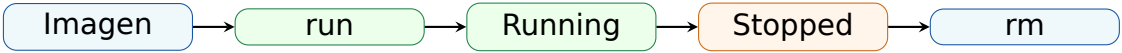
# Ejecutar
docker run --name mi-app -p 8080:8080 mi-app:1.0

# Ver contenedores
docker ps
docker ps -a

# Detener y eliminar
docker stop mi-app
docker rm mi-app
```

- `-t`: asigna nombre y etiqueta.
- `-p`: publica un puerto del contenedor en el host.
- El punto final indica el contexto de construcción.

10. Ciclo de vida de un contenedor



- `docker start/stop/restart/logs/exec` son operaciones habituales.
- El contenedor es reemplazable; los datos que necesiten persistir deben almacenarse de forma adecuada.

11. Puertos, variables y volúmenes

```
docker run -p 8080:80 \{}\{}
-e APP_ENV=dev \{}\{}
-v app-data:/var/lib/app \{}\{}
mi-app:1.0
```

- **Puertos:** conectan servicios del contenedor con el host.
- **Variables de entorno:** parametrizan la ejecución.
- **Volúmenes:** permiten conservar datos independientemente del contenedor.

No introduzcas contraseñas, tokens o claves privadas directamente en el `Dockerfile` ni en la imagen.

12. Redes Docker



- Los servicios pueden comunicarse mediante redes Docker.
- En Compose, los servicios de una misma aplicación pueden descubrirse por nombre de servicio.
- La red ayuda a separar componentes y reducir exposición innecesaria.

13. Docker Compose: varios servicios

- Compose permite definir y ejecutar aplicaciones formadas por varios contenedores.
- La configuración se expresa normalmente en un fichero `compose.yaml`.
- Es especialmente útil para desarrollo, pruebas e integración.

```
services:
web:
build: .
ports:
- "8080:8080"
depends_on:
- db
db:
image: postgres:16
```

14. Comandos esenciales de Compose

```
# Crear / arrancar servicios
docker compose up -d

# Construir de nuevo
docker compose up -d --build

# Estado
docker compose ps

# Logs
docker compose logs -f

# Parar y eliminar recursos definidos
docker compose down
```

La ventaja didáctica es que el entorno completo queda descrito como código y puede compartirse junto con el proyecto.

15. Mini proyecto Docker para alumnos

1. Crear una pequeña API o aplicación web.
2. Añadir un `Dockerfile`.
3. Construir una imagen versionada.
4. Ejecutar el contenedor y probar el endpoint.
5. Añadir un segundo servicio, por ejemplo PostgreSQL.
6. Definir ambos servicios en `compose.yaml`.
7. Documentar los comandos en el README.

Entrega: repositorio GitHub con código + Dockerfile + Compose + README + evidencias de ejecución.

16. ¿Qué significa CI/CD?

- **CI — Continuous Integration:** integrar cambios frecuentemente y validar automáticamente el código.
- **CD — Continuous Delivery:** mantener el software preparado para ser desplegado.
- **CD — Continuous Deployment:** automatizar además el despliegue cuando se cumplen las condiciones definidas.

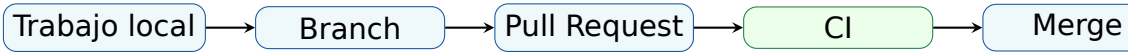
Código → **Build** → **Test** → **Package** → **Deploy**

17. ¿Por qué automatizar?

- Detectar errores antes de llegar a producción.
- Repetir las mismas comprobaciones en cada cambio.
- Reducir tareas manuales y pasos olvidados.
- Obtener evidencias: logs, resultados de tests, artefactos y estados del pipeline.
- Crear un proceso común para todo el equipo.

CI/CD no significa “desplegar sin control”: significa automatizar controles y hacer explícitas las condiciones para avanzar.

18. GitHub como plataforma de colaboració



- Git aporta control de versiones.
- GitHub aporta repositorios, colaboración y automatización mediante Actions.
- Un Pull Request puede convertirse en el punto de entrada de las pruebas automáticas.

19. GitHub Actions: el motor de CI/CD

- GitHub Actions permite automatizar compilación, pruebas y despliegue.
- Los workflows se almacenan en `.github/workflows/`.
- Se ejecutan en respuesta a eventos como `push` o `pull_request`.
- Cada workflow contiene uno o varios jobs; cada job contiene steps.

Para el alumno, la idea clave es sencilla: **un cambio en GitHub puede disparar automáticamente una secuencia de comprobaciones**.

20. Anatomía de un workflow

```
name: CI

on:
push:
pull_request:

jobs:
test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- name: Configurar entorno
run: echo "Preparando entorno"
- name: Ejecutar pruebas
run: pytest
```

- `on`: cuándo se ejecuta.
- `jobs`: qué trabajos se ejecutan.
- `steps`: acciones o comandos dentro de cada trabajo.

21. Primer pipeline CI para un proyecto Pyt

```
name: Python CI

on: [push, pull_request]

jobs:
test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-python@v5
with:
python-version: '3.12'
- run: pip install -r requirements.txt
- run: pytest
```

El pipeline debe fallar si las pruebas fallan. El estado del workflow se convierte en una señal objetiva para el equipo.

22. Integrar Docker dentro de CI

```
steps:
- uses: actions/checkout@v4

- name: Build Docker image
run: docker build -t mi-app:${{ github.sha }} .

- name: Run tests
run: docker run --rm mi-app:${{ github.sha }} pytest
```

- El mismo `Dockerfile` se utiliza en el entorno local y en CI.
- `github.sha` permite asociar la imagen a un commit concreto.
- El objetivo es detectar errores antes de publicar la imagen.

23. Publicar una imagen en un registry

Código → Build → Test → Registry → Deploy

```
# Ejemplo conceptual
docker build -t ghcr.io/usuario/mi-app:1.0 .
docker push ghcr.io/usuario/mi-app:1.0
```

- Un registry actúa como repositorio de imágenes.
- Conviene utilizar etiquetas reproducibles y evitar depender únicamente de `latest`.
- En un proyecto real, la publicación debe estar protegida por credenciales y permisos adecuados.

24. CD: del artefacto al despliegue



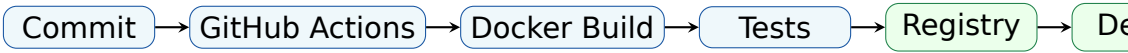
- El despliegue puede ser automático o requerir una aprobación.
- Es importante separar entornos y controlar qué versiones llegan a cada uno.
- La estrategia debe incluir rollback o recuperación ante fallos.

25. Secretos y seguridad en GitHub Actions

- No almacenar contraseñas, tokens o claves privadas en el repositorio.
- Utilizar secretos y variables configurados para el entorno adecuado.
- Aplicar mínimo privilegio a tokens y permisos.
- Evitar imprimir secretos en logs.
- Revisar acciones de terceros antes de utilizarlas en proyectos reales.

Un pipeline tiene acceso a código, artefactos y, potencialmente, credenciales. Por tanto, también debe considerarse un componente de seguridad.

26. Docker + GitHub Actions: flujo complet



- GitHub conserva el código y orquesta el proceso.
- Docker proporciona un artefacto ejecutable y reproducible.
- CI valida; CD promueve una versión validada hacia el siguiente entorno.

27. Pruebas dentro del pipeline

- Pruebas unitarias: comprobar funciones y componentes aislados.
- Pruebas de integración: comprobar comunicación entre componentes.
- Análisis estático y calidad del código.
- Pruebas de la imagen Docker: construcción, arranque y comportamiento esperado.
- Pruebas de seguridad como parte del proceso cuando el proyecto lo requiera.

El pipeline no debe limitarse a “compilar”: debe aportar evidencias de que el cambio cumple los criterios definidos.

28. Actividad práctica 1: Dockerizar una ap

1. Clonar un repositorio de ejemplo.
2. Identificar cómo se ejecuta actualmente.
3. Crear el `Dockerfile`.
4. Construir la imagen.
5. Ejecutar el contenedor.
6. Probar la aplicación desde el navegador o mediante `curl`.
7. Documentar el proceso.

Pregunta para el alumno: **¿qué dependencias o configuraciones necesita mi aplicación para funcionar fuera de mi ordenador?**

29. Actividad práctica 2: crear un CI con Gi

1. Crear el directorio `.github/workflows/`.
2. Crear `ci.yml`.
3. Configurar el evento `push` y `pull_request`.
4. Instalar dependencias.
5. Ejecutar las pruebas.
6. Provocar intencionadamente un fallo.
7. Corregirlo y comprobar que el pipeline vuelve a verde.

Resultado esperado: el alumno debe saber localizar el workflow, leer sus logs y explicar por qué ha pasado o fallado.

30. Actividad práctica 3: Docker + CI

1. Construir la imagen Docker desde GitHub Actions.
2. Ejecutar pruebas sobre la imagen.
3. Etiquetar la imagen con el commit.
4. Publicarla en un registry de prueba.
5. Comprobar que la imagen puede descargarse y ejecutarse.
6. Registrar evidencias del pipeline.

Portfolio: repositorio + Dockerfile + workflow + README + captura o enlace de ejecución + explicación de los pasos.

31. Errores frecuentes y cómo diagnosticar

- “Funciona localmente, falla en Docker” → revisar dependencias, rutas y variables.
- “El contenedor termina” → revisar el proceso principal y sus logs.
- “No puedo acceder al servicio” → revisar puerto, bind y red.
- “CI falla pero local funciona” → comparar versiones, entorno y comandos.
- “No aparece el workflow” → comprobar ubicación y sintaxis YAML.

```
docker logs <container>
docker inspect <container>
docker compose logs
```

32. Checklist final del alumno

- ☐ Sé explicar imagen vs. contenedor.
- ☐ Sé crear y ejecutar un `Dockerfile`.
- ☐ Sé utilizar puertos, variables y volúmenes.
- ☐ Sé definir varios servicios con Compose.
- ☐ Entiendo CI, Continuous Delivery y Continuous Deployment.
- ☐ Sé crear un workflow básico de GitHub Actions.
- ☐ Sé leer logs y diagnosticar un fallo del pipeline.
- ☐ Sé construir una imagen Docker desde CI.
- ☐ Sé explicar dónde deben gestionarse los secretos.

Objetivo final: pasar de **“ejecuto mi aplicación manualmente”** a **“tengo un proceso reproducible y automatizado para validarla”**.

33. Recursos oficiales y siguientes pasos

- Docker — documentación: https://docs.docker.com/
- Docker — Dockerfile: https://docs.docker.com/build/concepts/dockerfile/
- Docker Compose: https://docs.docker.com/compose/
- GitHub Actions — inicio rápido: https://docs.github.com/es/actions/get-started/quickstart
- GitHub Actions — sintaxis de workflows: https://docs.github.com/es/actions/reference/workflows-and-actions/workflow-syntax

Siguiente paso recomendado: construir un proyecto pequeño, contenerizarlo, añadir pruebas y automatizar el proceso con GitHub Actions.